

ReFEree: Reference-Free and Fine-Grained Method for Evaluating Factual Consistency in Real-World Code Summarization

ACL 2026

Suyoung Bae, CheolWon Na, Jaehoon Lee, Yumin Lee, YunSeok Choi, Jee-Hyong Lee

Sungkyunkwan University, South Korea

{sybae01, ncw0034, hoon1223, totoandy, ys.choi, john}@skku.edu

Presenter : Suyoung Bae

sybae01@skku.edu

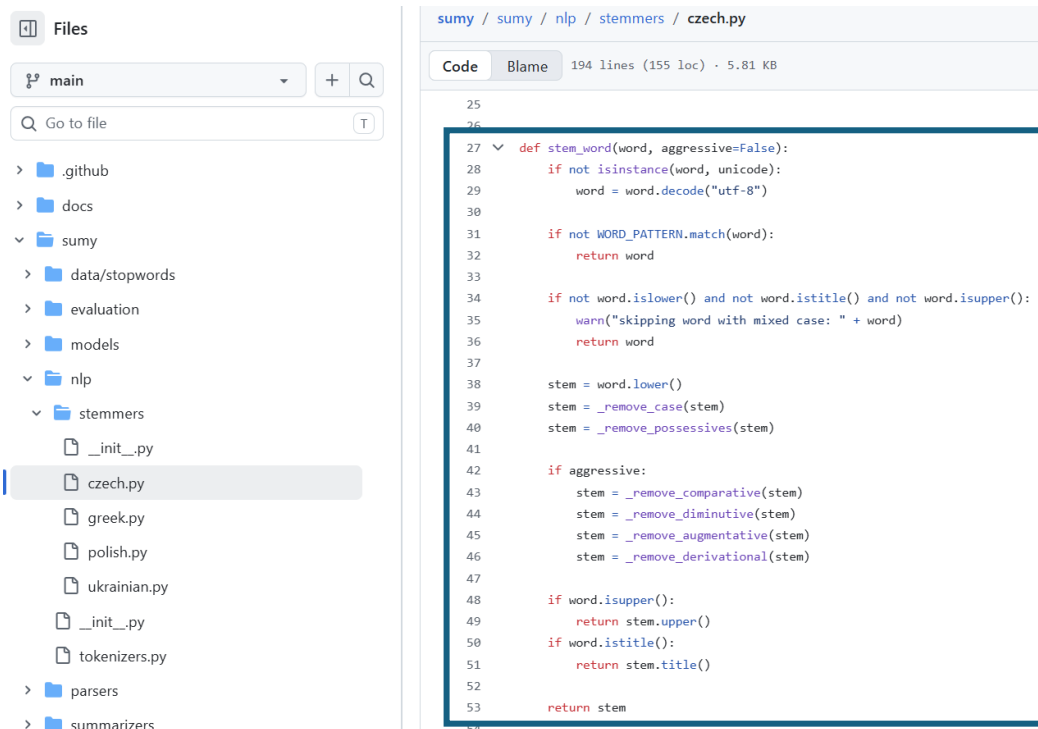
Information & Intelligence Lab

Content

- Introduction
- Proposed Method
- Experimental Results
- Conclusion

Repository-Level Code Documentation

- Repository-Level Code Documentation is a **long and descriptive summaries for functions, methods, or classes within a repository**, where each target entity is documented not only as a standalone unit but also in terms of its **inter-relations with other files, functions, and external information across the repository**.



Document the “stem_word” function.

Description: This function performs Czech word stemming with optional aggressive morphological processing. This function serves as the main entry point for Czech text stemming, applying basic case removal and possessive ending stripping by default, ...

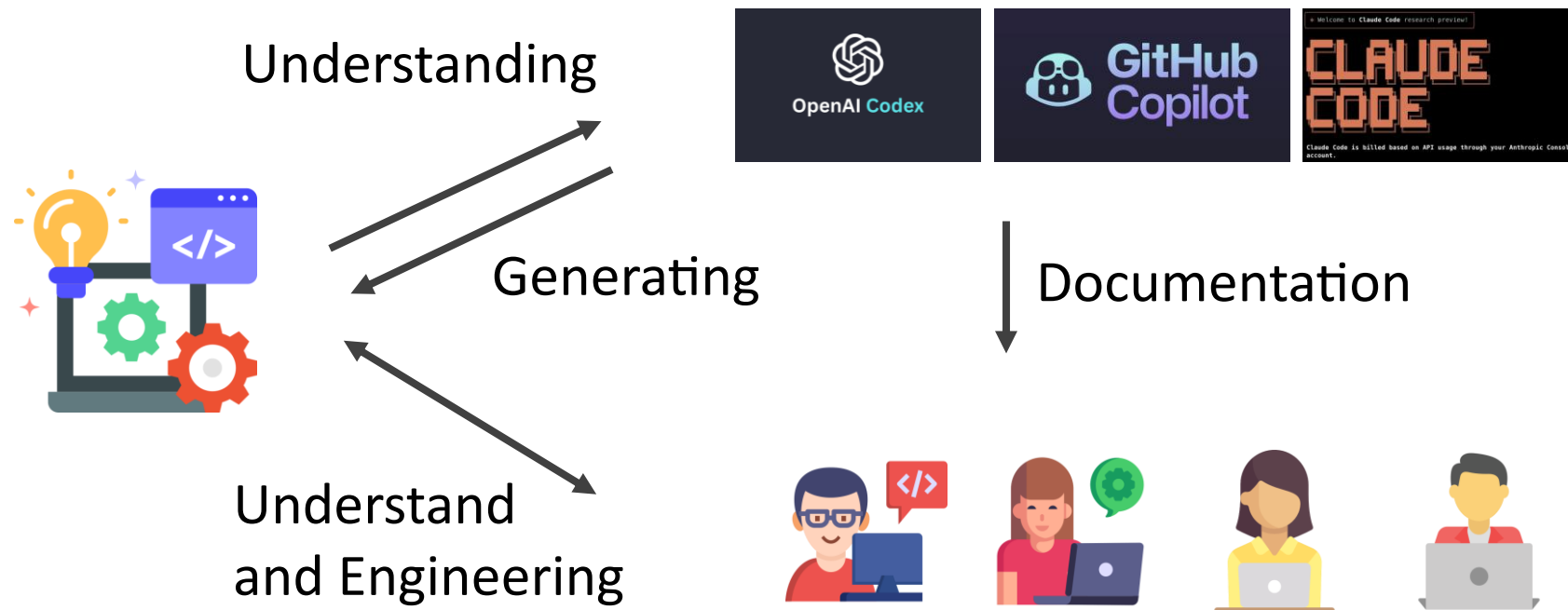
Args: word(str) A Czech word to be stemmed, which ...

Returns: str: The stemmed version of the input word, ...

Raises: None explicitly raised, though mixed-case ...

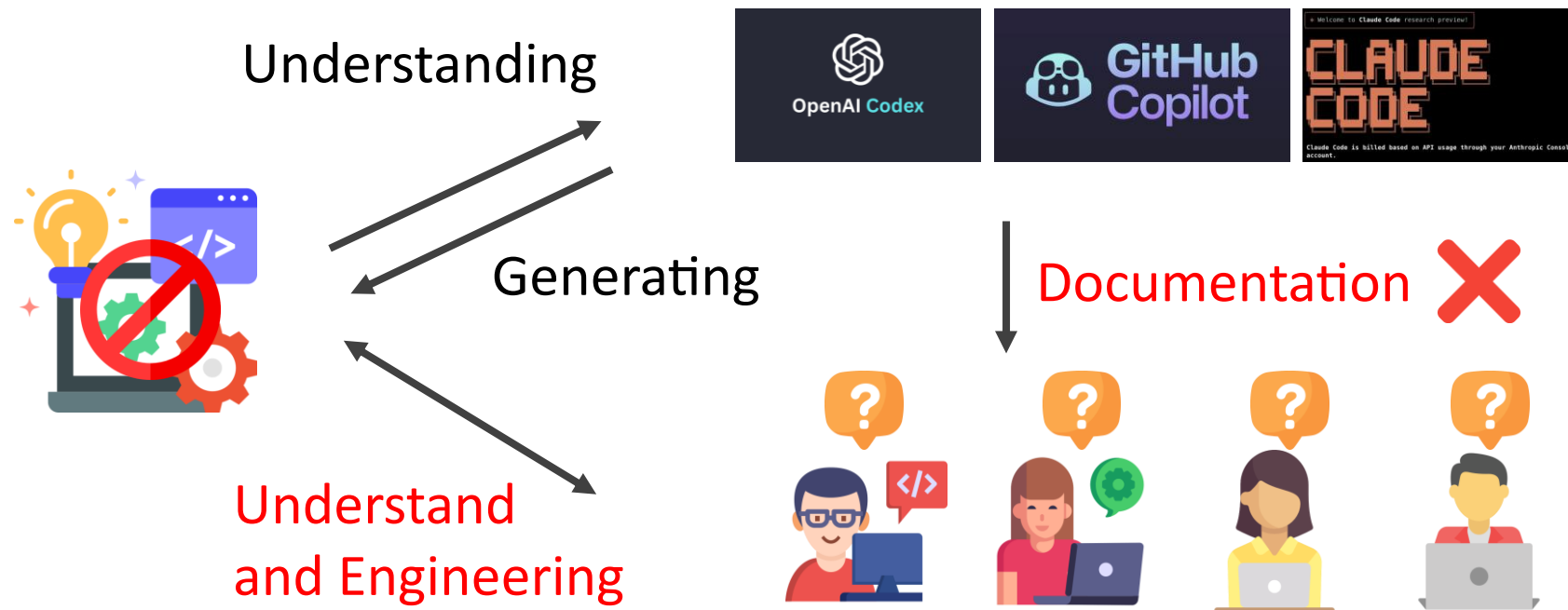
Repository-Level Code Documentation

- **Repository-level code documentation** are increasingly integrated into real-world development workflows to assist engineers in understanding and reviewing code.



Repository-Level Code Documentation

- However, when the generated summary does not accurately reflect the code's actual implementation, it can cause developers to **misunderstand the code**, leading to **delayed debugging** and **increased maintenance costs**.



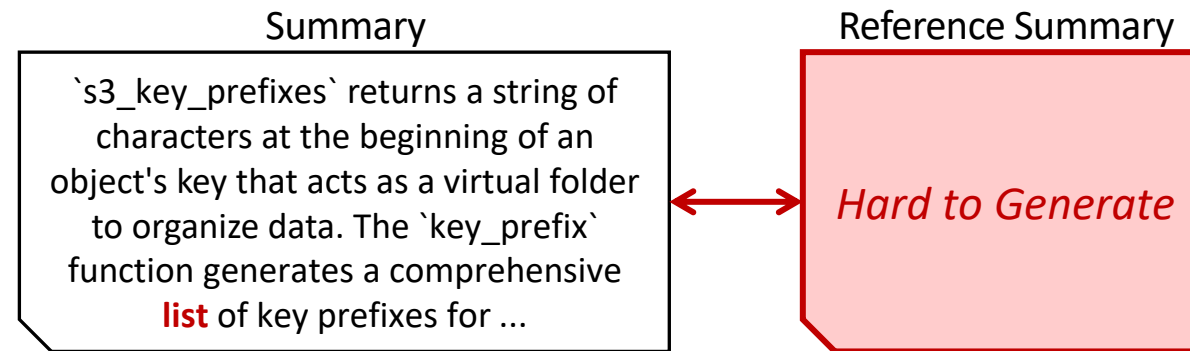
Factual Consistency in Repository-Level Code Documentation

- Therefore, it is important to accurately and reliably evaluate the factual consistency of LLM-generated code documents.
- **Factual Consistent Code Summary in Repository-Level:**
 1. Context should be consistent with the **Input code context**,
 2. But also consistent with **external information (= Information related to the input code)**

Problems in Existing Evaluation Methods

Existing evaluation methods do not correlate with human judgement.

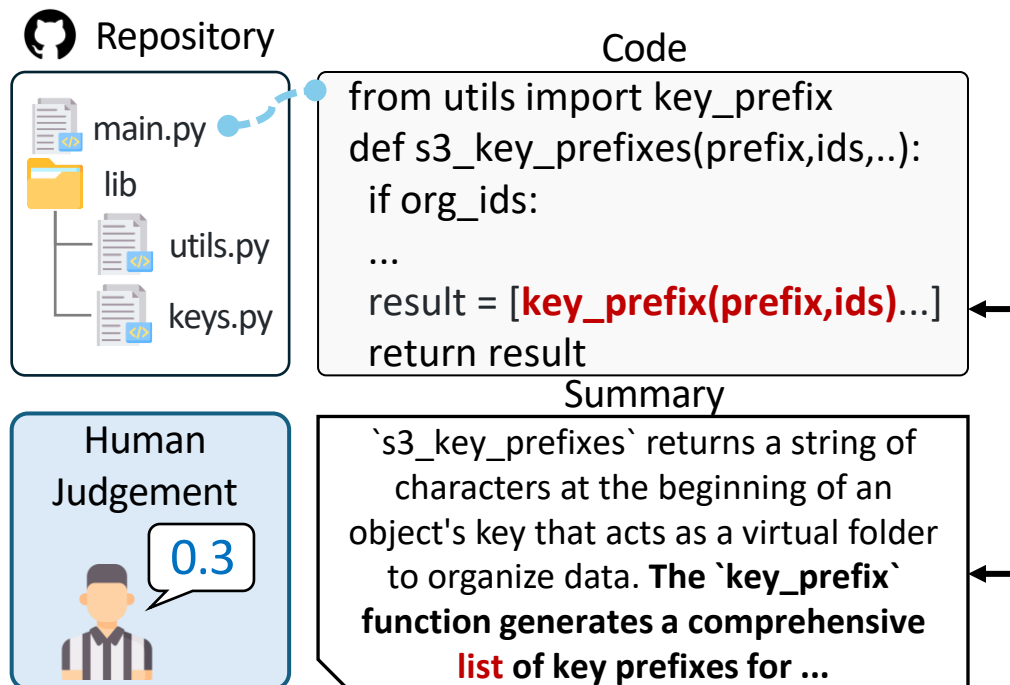
- Reference-based methods (ex: ROUGE, BLEU, BERTScore, ..) :
 - **Hard to generate** human-written references since Code2NL evaluation is a one-to-many problems.



Problems in Existing Evaluation Methods

Existing evaluation methods do not correlate with human judgement.

- **Reference-free methods (ex: LLM Judge Systems):**
 - Evaluate code summaries based solely on the input code **without considering external contexts.**
 - **Cannot provide fine-grained assessments** of inconsistencies in long summaries.



- 1 **Cannot provide evidence** without searching for the “key_prefix” function **defined in another file.**
- 2 **Treat the code summary as a whole**, failing to identify which specific sentences are inconsistent or to explain the underlying reasons for those inconsistencies.



We need ...

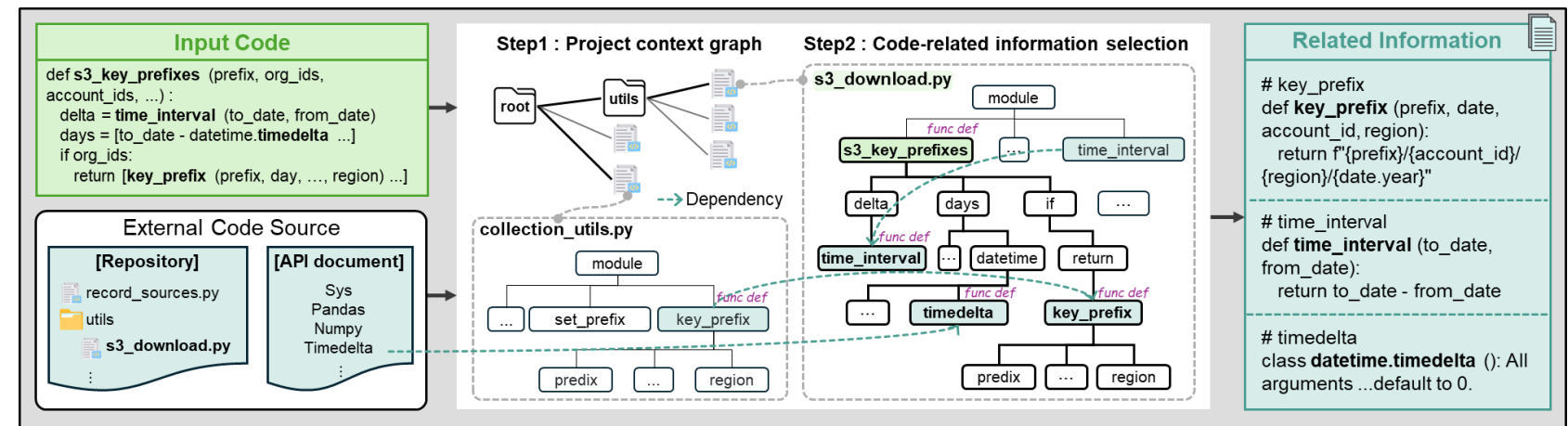
1. **We need to define four factual inconsistency criteria (C1-C4)**
2. **We propose two-stage evaluation pipeline** : reference-free method by first **searching input code-related information** and then **calculating the score via fine-grained evaluation**.

Proposed Method : ReFEree

See details in Section 3

Reference free and fine-grained method for evaluating the factual consistency of code summarization in a real-world environment

2 Searching input code related information



1 Defining four factual inconsistency criteria

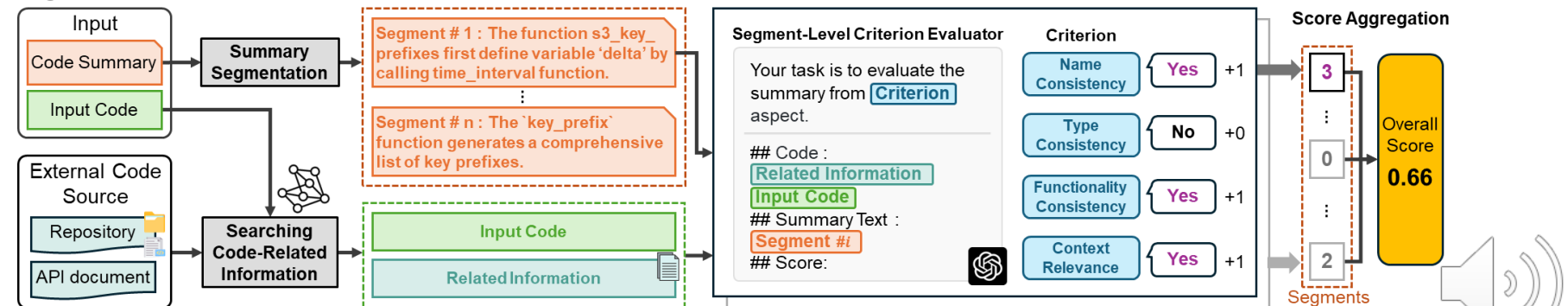
C1. Name Consistency

C2. Type Consistency

C3. Functionality Consistency

C4. Context Relevance

3 Calculating the overall score via fine-grained evaluation



Proposed Method : ReFEree

See details in Section 3

① Defining four factual inconsistency criteria

- Sample 100 real-world Python and Java functions from popular repositories
- Generate descriptive summaries using three LLMs
- Human annotators review all 300 summaries and manually **annotate factual inconsistencies**

Criteria (Distribution)	Definition
[C1] Name Inconsistency (14%)	The name of a function, class, or variable in the summary does not match the actual identifier in the input code or mistakenly refers to a different entity with the same name in the project.
[C2] Type Inconsistency (15%)	The described function's return type or variable type in the summary is inconsistent with the actual type in the input code or in directly dependent functions.
[C3] Functionality Inconsistency (35%)	The functionality or purpose described in the summary does not accurately reflect what the input code or its dependent functions actually implement. This often arises when dependency relationships are ignored or misinterpreted, leading to an incorrect description of behavior.
[C4] Context Irrelevant (33%)	The summary contains content that is unnecessary or irrelevant to the input code or relevant information, such as descriptions of unrelated entities or overly generalized dependency context that does not contribute to understanding the function.

Table 1: Four factual inconsistency criteria with the proportion of each for evaluating real-world code summarization.



Proposed Method : ReFEree

See details in Section 3

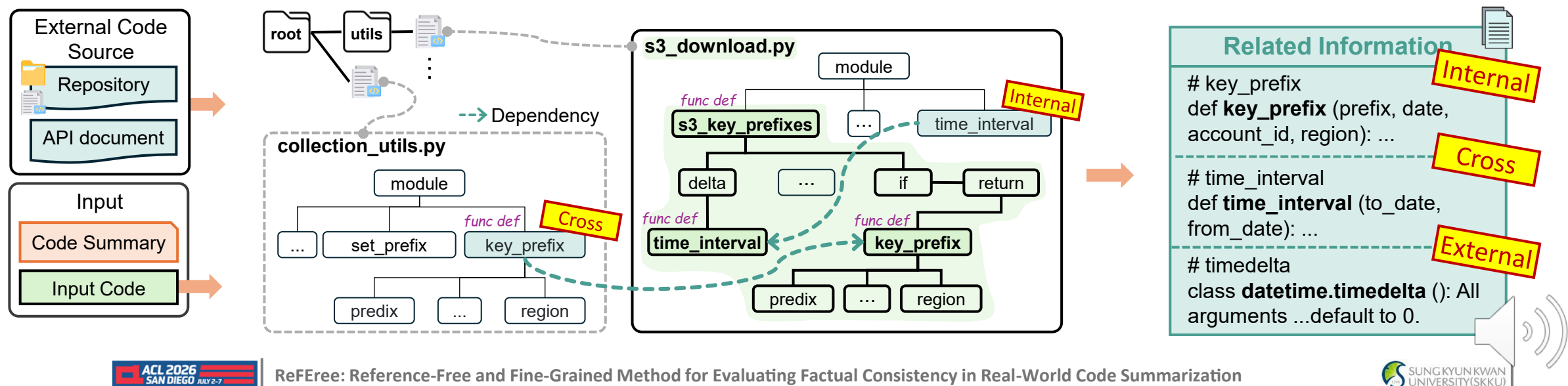
② Searching input code related information

• Step 1: Project context graph construction

- Traverse the **AST** of each file in the repository to collect the code entities
- Construct heterogeneous **DAG**.

• Step 2: Code-related information selection

- **DFS** on the project context graph
- **Crucial entity selection** and **1-hop dependency searching** to selectively search related information.

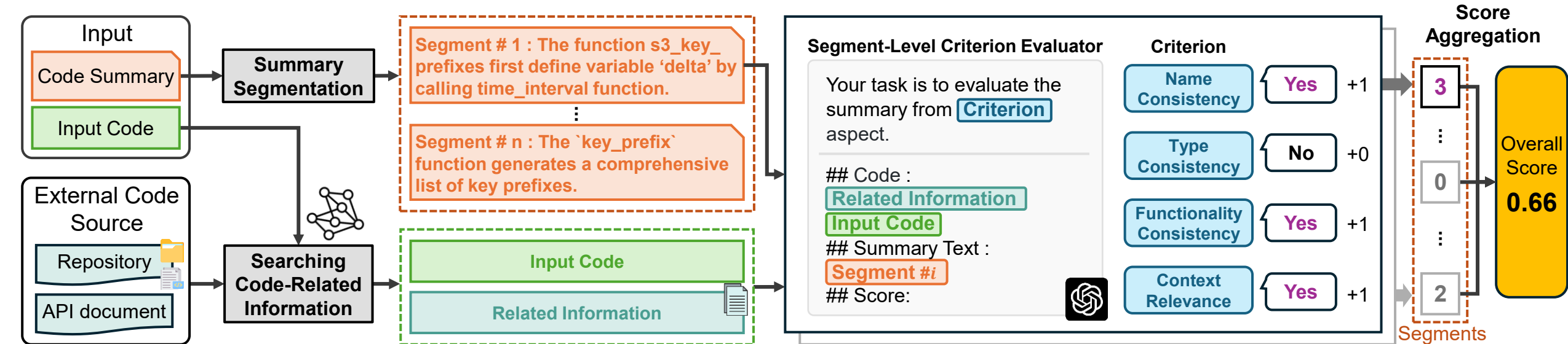


Proposed Method : ReFEree

See details in Section 3

③ Calculating the overall score via fine-grained evaluation

- Use NLTK sentence tokenizer to split a code summary into sentence-level segments
- Evaluate each segment against four criteria



$$SCORE = \frac{1}{|\mathcal{D}| \times |Criteria|} \sum_{S \in \mathcal{D}} \sum_{C \in Criteria} f(S, C)$$



Experimental Setups : Evaluation Benchmarks

See details in Section 4.1

- Construct a reliable set of human labels for factual consistency at both the summary and segment levels.
 - 1,825 functions from 115 Python repositories
 - 230 functions from 10 Java repositories

Input Code	Generated Summary	Summary Label
## Python Input Code ## <pre>def iter_slice(iterable: bytes, n: int): start = 0 stop = start + n final_offset = len(iterable) while True: if start >= final_offset: break rv = iterable[start:stop] start = stop stop = start + n yield rv, start >= final_offset</pre>	The function iter_slice is designed to process a bytes iterable by slicing it into fixed-size chunks of length n and yielding each chunk along with a boolean flag indicating whether the end of the iterable has been reached. Internally, it maintains a pair of indices, start and stop, which are used to extract the current slice from the iterable. After each yield, these indices are updated to prepare for the next slice. Notably, the function is named slice_iterator, reflecting its role in iterating over slices of data (C1, C3, C4). This approach is particularly useful in scenarios such as streaming data processing or chunked file reading, where handling large byte sequences in manageable pieces is essential (C3, C4). The boolean flag returned alongside each chunk helps client code determine when to stop requesting more data.	4

Related Information
 # java.lang.String #
 java.lang.String: The String class represents character strings in Java. All string literals are implemented as instances of this class. Strings are constant (immutable); their values cannot be changed after creation. The class provides methods for examining characters, comparing strings, searching, extracting substrings, and case conversions.

StringUtils.isEmpty #
 public static boolean isEmpty(final String str){
 return str == null || str.length() == 0;
 }

 # StringUtils.INDEX_NOT_FOUND #
 public static final int INDEX_NOT_FOUND = -1;

Java Input Code
 public static int lastIndexOf(String str,String searchStr){
 if (StringUtils.isEmpty(str)) {
 return StringUtils.INDEX_NOT_FOUND;
 }
 return str.lastIndexOf(searchStr);
 }

The 'lastIndexOf' function is a utility method that finds the last occurrence of a given substring within a string. It utilizes the 'StringUtils' class to first check if the input string is empty using the 'isEmpty' method, which is actually not a real method in the 'StringUtils' class (**C1, C2, C3, C4**). If the string is empty, it returns a constant 'INDEX_NOT_FOUND' from the 'StringUtils' class, which is correctly defined as '-1'. This function is particularly useful in data processing and parsing scenarios where locating the position of a substring is critical (**C4**). Additionally, the method is often employed in web development for URL parsing and manipulation, though this context is not directly related to the core functionality of the code (**C3, C4**).

3



Experimental Results : Main Comparisons

See details in Section 5.1

- ReFEree outperforms all baselines and demonstrates strong and stable alignment with human judgment, with most data points closely following the ideal correlation line.

Methods	Python				Java			
	r_p	r_s	τ	Average	r_p	r_s	τ	Average
<i>reference-based methods</i>								
ROUGE-1 (Lin, 2004)	0.046	0.044	0.034	0.041	0.201	0.186	0.140	0.176
ROUGE-2 (Lin, 2004)	0.030	0.031	0.024	0.028	0.194	0.176	0.137	0.169
ROUGE-L (Lin, 2004)	0.045	0.037	0.029	0.037	0.202	0.178	0.137	0.172
BLEU (Papineni et al., 2002)	0.016	0.018	0.015	0.017	0.081	0.065	0.058	0.068
METEOR (Banerjee and Lavie, 2005)	0.033	0.024	0.019	0.025	0.135	0.139	0.105	0.126
BERTScore (Zhang et al., 2020)	0.015	0.000	0.000	0.005	0.189	0.149	0.113	0.150
SBCS (Reimers and Gurevych, 2019)	0.068	0.036	0.028	0.044	0.363	0.227	0.178	0.256
SBED (Reimers and Gurevych, 2019)	-0.058	-0.036	-0.028	-0.041	-0.348	-0.227	-0.178	-0.251
<i>reference-free methods</i>								
SIDE (Mastropaolo et al., 2024)	-0.064	-0.056	-0.043	-0.054	0.032	0.024	0.019	0.025
LLM-judge (Zheng et al., 2023)	0.419	0.410	0.360	0.396	0.385	0.363	0.318	0.355
G-Eval (Liu et al., 2023)	0.427	0.413	0.360	0.400	0.457	0.407	0.355	0.406
Factscore (Min et al., 2023)	0.410	0.426	0.338	0.391	0.391	0.365	0.310	0.355
CODERPE (Wu et al., 2024)	0.418	0.405	0.353	0.392	0.457	0.398	0.347	0.401
ReFEree (w/o info)	0.432	0.432	0.349	0.404	0.469	0.458	0.387	0.438
ReFEree (w/ info)	0.497	0.489	0.390	0.459	0.515	0.502	0.423	0.480

Table 2: We compare our method (ReFEree) with 8 reference-based and 5 reference-free methods using Pearson (r_p), Spearman (r_s), and Kendall’s Tau (τ) correlation coefficients at the summary level. Our method achieves p -values all below 0.005, indicating statistical significance. The best result is shown in **bold**.

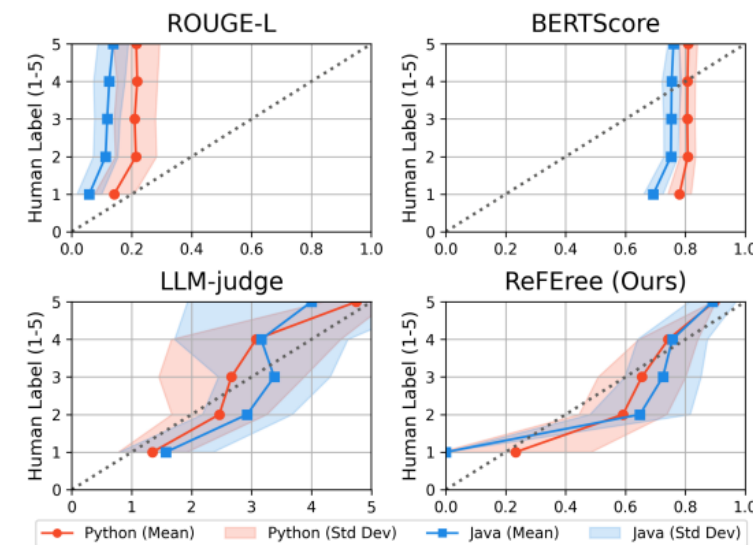


Figure 3: Correlation between human judgment and the scores from each method. Each plot shows the relationship between scores obtained from each method (x-axis) and human labels (y-axis), with red for Python and blue for Java. The gray dashed line represents perfect correlation. Since human labels range from 1 to 5, there are no values at $y = 0$.



Experimental Results : Further Analysis

See details in Section 5.2, 5.3

- [Table 3] ReFEree achieves reliable detection of segment-level factual inconsistencies.
- [Table 5] It also shows that evaluating all four inconsistency criteria in real-world code summaries produces the most reliable assessment

Criteria	C1	C2	C3	C4	Average
Python	0.922	0.961	0.931	0.924	0.934
Java	0.913	0.991	0.940	0.876	0.930

Table 3: Accuracy of segment-level factual inconsistency evaluation. We report the accuracy of *ReFEree* in identifying four criteria (C1–C4) defined in Section 3.1.

Methods	Python	Java
G-Eval	0.400	0.406
ReFEree (w/ C1)	0.394 (-1.6%)	0.318 (-5.2%)
ReFEree (w/ C2)	0.417 (+4.3%)	0.379 (-6.8%)
ReFEree (w/ C3)	0.419 (+4.8%)	0.391 (+12.7%)
ReFEree (w/ C4)	0.405 (+1.2%)	0.352 (+3.1%)
ReFEree (w/ all)	0.459 (+14.6%)	0.480 (+18.1%)

Table 5: Average correlation coefficients (Pearson, Spearman, Kendall’s Tau) for *ReFEree* (w/ all), its single-criterion ablations (w/C1–C4), and G-Eval. Percentages indicate relative performance change compared to G-Eval.



Experimental Results : Generalizability and Stability

See details in Section 5.4, 5.5

- [Table 6] It achieves consistent performance improvements across different LLM evaluator settings.
- [Table 7] *ReFEree* provides more stable and reliable assessments than conventional LLM-based methods

Models	Methods	Python	Java
Llama3 (8B)	G-Eval	0.145	0.217
	G-Eval (w/ info)	0.171	0.251
	ReFEree	0.187	0.367
Mistral (7B)	G-Eval	0.191	0.196
	G-Eval (w/ info)	0.191	0.242
	ReFEree	0.233	0.250
Qwen2 (7B)	G-Eval	0.221	0.185
	G-Eval (w/ info)	0.243	0.238
	ReFEree	0.284	0.304
Qwen2.5-Coder (7B)	G-Eval	0.247	0.243
	G-Eval (w/ info)	0.304	0.348
	ReFEree	0.351	0.394
GPT4o-mini	G-Eval	0.242	0.390
	G-Eval (w/ info)	0.259	0.428
	ReFEree	0.354	0.429
GPT4.1-mini	G-Eval	0.400	0.406
	G-Eval (w/ info)	0.418	0.463
	ReFEree	0.459	0.480

Table 6: Generalizability of *ReFEree* Across LLMs. We compare the performance improvements of *ReFEree* across six different LLMs, relative to G-Eval and G-Eval (w/ info) for the average of Pearson, Spearman, and Kendall’s Tau correlation coefficients ($p < 0.005$).

Settings	Different Prompts		Different Seeds	
Methods	G-Eval	ReFEree	G-Eval	ReFEree
Python	0.683	0.733	0.805	0.893
Java	0.400	0.730	0.712	0.901

Table 7: Stability of *ReFEree*. We compute inter-annotator agreement (IAA) scores across three independent evaluation results for each method (G-Eval and ReFEree) when evaluated under different prompts or different seed settings, for both G-Eval and ReFEree.

Conclusions

- We introduced a novel **reference-free** and **fine-grained** method for assessing the factual consistency of real-world code summarization.
- We also **constructed a high-quality human-labeled benchmark** to validate the reliability of the proposed method.
- The comprehensive evaluation results demonstrate that ReFEree achieves **high accuracy, reliability, generalizability, and stability**, making it effectively applicable to real-world code summarization tasks.

Thank You

sybae01@skku.edu